VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Multidisciplinary Project

H.E.R.A

(Home Environment & Response Assistant)

Instructor: Dr. Võ Thị Ngọc Châu

Student's Name	Student ID
Nguyễn Tiến Hưng	2352441
Hồ Lâm Khánh Vy	2353353
Trần Anh Đức	2352271
Nguyễn Khánh Hưng	2352435

Ho Chi Minh City, January 2026.

Member list & Workload

Group Leader: Nguyễn Tiến Hưng - hung.nguyenk23cse@hcmut.edu.vn

No.	Full name	Contribution	Percentage
1.	Nguyễn Tiến Hưng		100%
2.	Hồ Lâm Khánh Vy		100%
3.	Trần Anh Đức		100%
4.	Nguyễn Khánh Hưng		100%

Lecturer's Assessment

No.	Full name	Evaluation	Notes
1.	Nguyễn Tiến Hưng		
2.	Hồ Lâm Khánh Vy		
3.	Trần Anh Đức		
4.	Nguyễn Khánh Hưng		

Contents

1 Introduction

1.1 Problem Statement

Context and Target Environment

The evolution of the "Smart Home" has transitioned from simple connectivity to the demand for intelligent automation. While the general market is saturated with fragmented IoT solutions, this project specifically targets modern urban households—such as mid-sized apartments or townhouses—occupied by busy families, elderly members, or pets. In these environments, standard smart platforms function merely as centralized remote controls. They enable manual actuation via smartphone applications but fail to deliver a cohesive, autonomous living experience tailored to the specific safety, convenience, and energy needs of an active urban family.

Key Challenges

Despite the ubiquity of connected hardware, several critical architectural and functional deficiencies persist in residential deployments:

- **Data Silos & Fragmentation:** Devices from different manufacturers often operate on proprietary protocols. This fragmentation prevents the aggregation of data into a single Unified Source of Truth, making holistic analytics and cross-device correlation impossible.[1]
- **Deterministic vs. Probabilistic Limitations:** Current systems rely on rigid, rule-based automation (e.g., "turn on lights at 6 PM"). They lack the probabilistic reasoning needed to adapt to dynamic human routines or distinguish between a human and a pet triggering a motion sensor.
- **Data Rich, Information Poor (DRIP):** IoT devices generate high-velocity telemetry daily. Without robust Data Engineering pipelines (ETL processes) to filter noise, users are overwhelmed with raw notifications rather than actionable insights regarding energy anomalies.
- **Reactive Operation & Energy Inefficiency:** Systems operate reactively (e.g., cooling a room only after it gets hot). This lag degrades user comfort and leads to significant energy waste, as the system cannot pre-emptively optimize resources.

Operational Assumptions & Constraints

To effectively address these challenges, the H.E.R.A system is designed with the following bounded parameters:

- **Physical Scope:** The system is optimized for a single enclosed residential room (approximately 15–30 square meters), such as a bedroom or living room. The spatial model focuses on localized environmental monitoring and occupant interaction within this bounded area to ensure precise sensor calibration, accurate spatial tracking, and simplified Digital Twin representation.
- **Infrastructure Compatibility:** The architecture assumes the presence of a stable Wi-Fi network for cloud-based AI processing (such as NLP and heavy predictive modeling). However, it requires local Edge processing capabilities (e.g., via ESP32) to maintain critical deterministic operations and sensor readings if internet connectivity drops.

- **Privacy & Security:** Given the continuous ingestion of environmental telemetry and voice commands, the system is constrained by strict data privacy requirements, ensuring that behavioral data and audio inputs are securely processed and shielded from unauthorized access.

The Need for a Solution and Scope of Intelligence

There is an urgent need for a comprehensive AIoT (AI + IoT) platform that bridges the gap between physical control and cognitive processing for urban homes. A truly intelligent system must transcend basic command execution to act as a proactive agent. H.E.R.A addresses this by narrowing its "smart" focus onto three prioritized pillars:

1. **AI-Enhanced Security:** Utilizing machine learning to differentiate between genuine human presence and pets, thereby eliminating false alarms and providing reliable monitoring.
2. **Proactive Energy Management:** Deploying predictive analytics to optimize power consumption based on historical usage patterns and environmental forecasts.
3. **Frictionless Interaction:** Leveraging Natural Language Processing (NLP) to allow residents to control their environment intuitively, effectively shifting the paradigm from manual "Smart Control" to autonomous "Smart Living."

1.2 Stakeholders and Key Users

To ensure the H.E.R.A system is designed with practical applicability, the project identifies two core user groups with distinct roles:

- **Primary Key Users (Residents/Family Members):** These are the end-users who reside in and interact daily with the smart home environment. Their primary concerns are comfort, convenience, and health.
- **Secondary Key Users (System's Administrator/Homeowner):** This is the decision-maker responsible for property maintenance, security assurance, and managing the operational costs of the household.

1.3 Business Requirements

Derived from the analysis of Key Users, the project establishes the core business requirements that the H.E.R.A system must fulfill:
BR1: For Residents (Primary Key Users)

- Business Objective: To enhance the quality of life, convenience, and health protection through an automated living environment that proactively understands the user.
- Pain Points: Users are fatigued by manually adjusting multiple applications for different devices; they desire a system that intuitively "understands" their preferences and routines.
- Needs: A truly hands-free experience where the home autonomously adjusts temperature and lighting based on occupancy and habits, alongside health monitoring via environmental sensors.

- Success Criteria: The system accurately processes unstructured language commands (via NLP) and autonomously executes environmental adjustments (e.g., lighting, HVAC) according to the context, completely eliminating the need for manual smartphone app interactions.

BR2: For the System's Administrator (Secondary Key Users)

- Business Objective: To optimize operational costs (specifically electricity consumption) and ensure robust, reliable security for the property.
- Pain Points: Persistent anxiety regarding rising electricity bills, potential security breaches, or fire hazards when away from home. Furthermore, the constant nuisance of false security alarms is a significant concern.
- Needs: Detailed analytical reports on energy consumption to identify waste, coupled with real-time, AI-filtered security alerts that accurately distinguish between false alarms (e.g., a pet moving) and genuine threats.
- Success Criteria: The provision of an interactive Dashboard displaying energy consumption metrics and forecasts; and the successful deployment of a Classification Model to differentiate human from pet movements, thereby significantly minimizing the rate of false alarm notifications.

1.4 Solution

To fulfill the aforementioned business requirements (BR1 & BR2), the H.E.R.A (Home Environment & Response Assistant) is a comprehensive AI-powered virtual assistant system designed to revolutionize residential living. Unlike traditional passive smart homes, H.E.R.A integrates a robust IoT infrastructure with advanced Natural Language Processing (NLP) and Data Analytics. This fusion allows the system to not only continuously monitor environmental conditions but also understand and execute complex user commands via voice or text. By bridging the gap between raw telemetry and human intent, H.E.R.A delivers a living environment that is proactively energy-efficient, secure, and deeply personalized.

System Workflow

The system operates on a dual-stream data pipeline, integrating both automated environmental monitoring and user-centric interaction into a unified architecture:

1. **Multi-modal Data Acquisition:** The system continuously ingests data from two primary sources:
    - Telemetry: Sensors monitor physical parameters (temperature, humidity, motion) and device states, transmitting data via MQTT.
    - User Interaction: A dedicated interface captures voice commands or text inputs from the user, serving as the direct communication bridge.
2. **Processing & Verification:** A centralized backend processes these incoming streams. For telemetry, it filters noise and verifies if data exceeds safety thresholds. Simultaneously, for user inputs, it initiates a Speech-to-Text (STT) process (if applicable) to convert audio signals into processable query strings.
3. **Intelligence Layer (NLP & Analytics):** This is the decision-making core:

- Natural Language Understanding (NLU): AI models analyze user queries to extract intents (e.g., "turn on") and entities (e.g., "living room light").
- Predictive Analytics: Machine learning algorithms analyze historical logs to forecast trends or detect anomalies, enriching the decision context.

4. **Action, Control & Feedback:** Based on either AI interpretation or threshold triggers, the system executes commands:
   - Device Control: Sending signals to microcontrollers (ESP32/YoLo) to adjust physical devices.
   - User Response: Generating a natural language response (Text-to-Speech) to confirm actions or report status back to the user.
5. **Activity Logging:** Every interaction—whether a sensor reading, a voice command, or a system automated response—is recorded in an activity log. This maintains a single source of truth for debugging, dashboard visualization, and future model training.

By combining reactive automation with interactive AI capabilities, this workflow fulfills the dual purpose of a robust monitoring system and an intelligent virtual assistant.

Architectural Hierarchy

To ensure scalability, maintainability, and a clear separation of concerns, the H.E.R.A system architecture is bifurcated into two distinct layers. This two-tier design decouples the physical hardware constraints from the high-level computational logic:

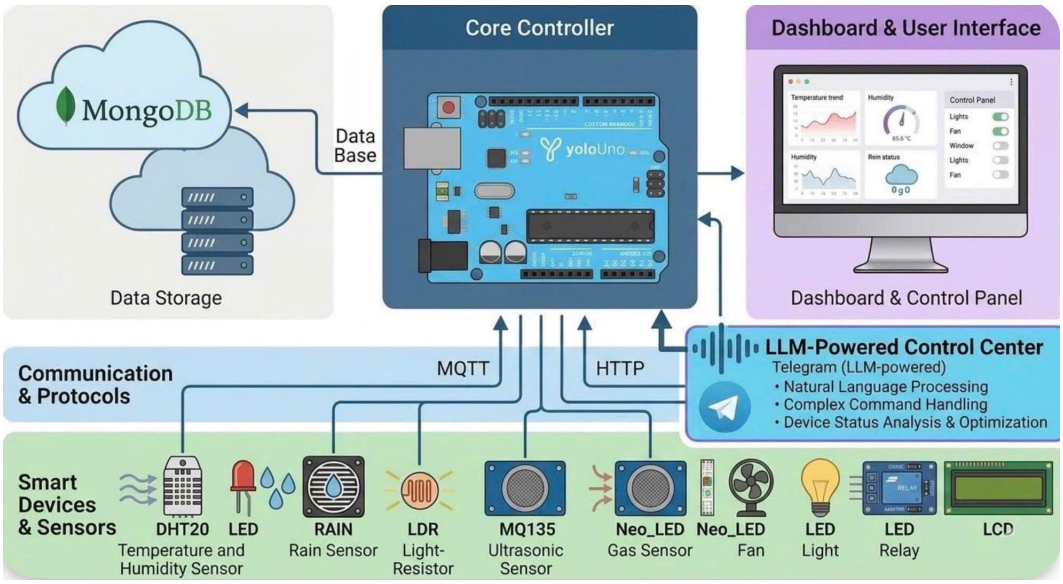

Figure 1.1: System Architecture of H.E.R.A

- **Layer 1: The Physical Perception & IoT Control Layer**
  This foundational layer serves as the direct interface with the physical environment. It is responsible for Data Acquisition and Device Actuation.

  - **Sensing & Ingestion:** A network of distributed sensors captures raw environmental telemetry (temperature, humidity, motion) and converts physical phenomena into digital signals. These signals are transmitted via lightweight protocols (MQTT) to ensure low-latency communication.
  - **Execution:** This layer receives operational commands from the upper layer and executes them through actuators (relays, IR blasters, dimmers) to physically alter the state of the home environment.
  - **Edge Responsibility:** It handles immediate hardware interactions and ensures connection stability, acting as the reliable "hands and eyes" of the system.

- **Layer 2: The Cognitive Intelligence & Orchestration Layer**
  Hosted on a central server, this high-level layer abstracts the complexity of decision-making. It focuses on Language Processing, Data Analysis, and Command Dispatching.
  - **Natural Language Understanding (NLU):** This module processes user inputs (text or voice), parsing unstructured language to extract intents and entities (e.g., converting "I'm cold" into a specific "increase temperature" command).
  - **Decision Engine:** Utilizing historical data and pre-trained AI models, this engine analyzes patterns to make autonomous decisions (e.g., anomaly detection for security or predictive energy saving).
  - **Orchestration:** acting as the central coordinator, it translates high-level decisions into specific, protocol-compliant commands and dispatches them down to the IoT layer for execution.

The interaction between these two layers creates a seamless feedback loop: the Lower Layer provides the context (data), while the Upper Layer provides the intelligence (actionable decisions).

1.5 Goal of the Project

Primary Objectives:

- Develop a Scalable IoT Architecture: Build a system capable of handling concurrent connections from multiple sensor nodes with low latency.
- Implement Data-Driven Automation: Move beyond simple timers to behavior-based automation using machine learning algorithms.
- Optimize Energy Consumption: Provide visualization and forecasting tools that empower users to reduce their carbon footprint and electricity costs.
- Enhance Security with AI: Utilize computer vision and anomaly detection techniques to provide superior security coverage compared to traditional motion sensors.

Scope of the Project:

- IoT Core Infrastructure: Implementation of essential smart home management features including device provisioning (registration), state synchronization (on/off status), real-time command dispatching via MQTT, and role-based access control (RBAC) for family members.
- Data Pipeline Integration: Development of automated ETL (Extract, Transform, Load) pipelines to ingest high-frequency sensor telemetry, clean noise, and structure data for analytical processing.

- AI-Driven Insights: Leveraging machine learning models to transform raw data into intelligence:
  - Regression Models: Forecasting energy consumption trends for the upcoming month.
  - Classification Models: Distinguishing between human presence and pets to filter false security alarms.
  - Clustering Algorithms: Grouping similar user activity patterns to recommend personalized automation schedules.
- Visualization & Reporting: Deployment of interactive dashboards featuring heatmaps (for room occupancy) and time-series charts to visualize historical environmental data and real-time system states.
- Digital Twin Integration & Simulation-to-Reality (Sim2Real):
  - Virtual Environment Replication: Development of a 3D digital twin of the home environment to synchronize and visualize the real-time status of IoT devices and environmental sensors.
  - Scenario Simulation: Creating "What-if" scenarios, such as fire outbreaks, gas leaks, or security breaches, to validate the responsiveness and accuracy of the system's emergency protocols without risking physical assets.
  - Sim2Real Framework: Ensuring that the control logic and AI models developed in the simulation can be seamlessly transitioned to physical hardware with minimal reconfiguration, facilitating robust testing and reducing the risk of equipment failure during deployment.
- Hardware & Architecture Scope: To establish precise physical boundaries and fulfill the required minimum number of input/output (I/O) interfaces, the system's hardware is distributed across two primary computational nodes:
  - Input Sensors: Integrates a Temperature/Humidity Sensor and a Light Sensor to continuously capture multi-modal environmental telemetry.
  - Output Actuators & Displays: Utilizes an LCD screen for real-time state visualization, alongside physical actuators including LED lights, a Mini Fan, and a Relay module to execute environmental adjustments and simulate appliance control.
- The Central Processing Node (PC): Serving as the cognitive intelligence hub (Layer 2), a computer system integrates a Microphone (Sound Sensor) as its primary input. This node is strictly dedicated to intensive audio acquisition and Speech-to-Text (STT) processing , subsequently feeding the Natural Language Understanding (NLU) pipeline before dispatching execution commands to the Edge Node via MQTT.

2 Multi-Agent AI Pipeline Architecture

The transition from a monolithic language-model controller to a *hierarchical multi-agent system* (MAS) constitutes the principal architectural upgrade of the H.E.R.A cognitive layer. This section formalises the theoretical underpinnings, describes the concrete software design, and justifies each engineering decision.

2.1 Motivation and Theoretical Background

2.1.1 Limitations of the Monolithic Single-Agent Approach

The initial H.E.R.A prototype employed a single large language model (LLM)—Qwen 2.5 at 7 billion parameters—to handle *every* user interaction: natural-language understanding (NLU), device control, sensor data interpretation, anomaly explanation, and general conversation. While functional, this architecture exhibits three well-documented deficiencies [2]:

1. **Latency inefficiency.** A 7 B-parameter model requires 2–5 s per inference on consumer hardware, even for trivial commands such as "turn on the LED." A smaller, task-specific model can resolve the same command in ∼300 ms.
2. **Prompt dilution.** A single system prompt must simultaneously encode rules for device control, sensor interpretation, anomaly analysis, and conversational etiquette. As the combined prompt grows, instruction-following accuracy degrades—a phenomenon termed *lost-in-the-middle* [3].
3. **Failure coupling.** A hallucination in the conversation sub-task (e.g. generating a fabricated image URL) can corrupt the response even when the device-control sub-task executed correctly.

2.1.2 Multi-Agent Systems in AI Literature

A multi-agent system is defined as a collection of autonomous agents that interact within a shared environment to achieve individual or collective goals [2]. Recent work in LLM-based MAS architectures—such as AutoGen [4], CrewAI, and LangGraph—demonstrates that decomposing complex reasoning into specialised sub-agents improves accuracy, reduces latency, and enhances modularity.
The H.E.R.A pipeline adopts the *Orchestrator–Specialist* pattern (a restricted form of the hierarchical MAS topology), wherein:

- A lightweight **Orchestrator** agent performs intent classification and routes messages to the appropriate specialist.
- Each **Specialist** agent owns a narrow domain, a focused system prompt, and (optionally) a dedicated set of tools.

2.1.3 Small Language Models as Efficient Agents

The recent proliferation of *Small Language Models* (SLMs)—models with 0.5–3 B parameters such as Qwen 2.5:1.5b, Phi-4-mini, and Gemma 2:2b—enables deployment of multiple specialised agents on the *same* hardware that previously hosted a single 7 B model.
Key advantages of SLMs in an agent context include:

- **Lower memory footprint:** A 1.5 B model (FP16) occupies ∼3 GB VRAM versus ∼14 GB for a 7 B model.
- **Faster time-to-first-token:** Reduced KV-cache computation yields sub-second latency for short prompts.
- **Sufficient capability for constrained tasks:** Intent classification and structured tool invocation do not require the broad world knowledge of a 7 B+ model [5].

Table 1 summarises the resource trade-offs.

Table 1: Comparison of LLM sizes for agent tasks.

Task	**Monolithic (7 B)**	**MAS (SLMs)**	**Speed-up**
Toggle LED	2–5 s	∼300 ms (1.5 B)	∼10×
Sensor query	2–5 s	∼800 ms (3 B)	∼4×
Anomaly analysis	2–5 s	∼1 s (3 B+rules)	∼3×
RAM footprint	∼6 GB	∼4 GB total	33% less

2.2 Software Architecture

2.2.1 Design Patterns

The pipeline is structured around three complementary design patterns:

1. **Hexagonal Architecture (Ports & Adapters)** [6]: the agent core is isolated from all I/O concerns. Input adapters (Telegram, Voice, REST) and output adapters (Telegram reply, TTS audio) plug into well-defined *ports*, allowing new interaction modalities to be added without modifying the agent logic.
2. **Mediator Pattern** [7]: the Orchestrator acts as a central mediator—agents never communicate directly with one another. This eliminates n^2 coupling and simplifies testing.
3. **Strategy Pattern** [7]: each specialist agent implements a common `AgentBase` interface. The Orchestrator selects the appropriate "strategy" (agent) at runtime based on the classified intent.

Figure 2.1 illustrates the layered architecture.

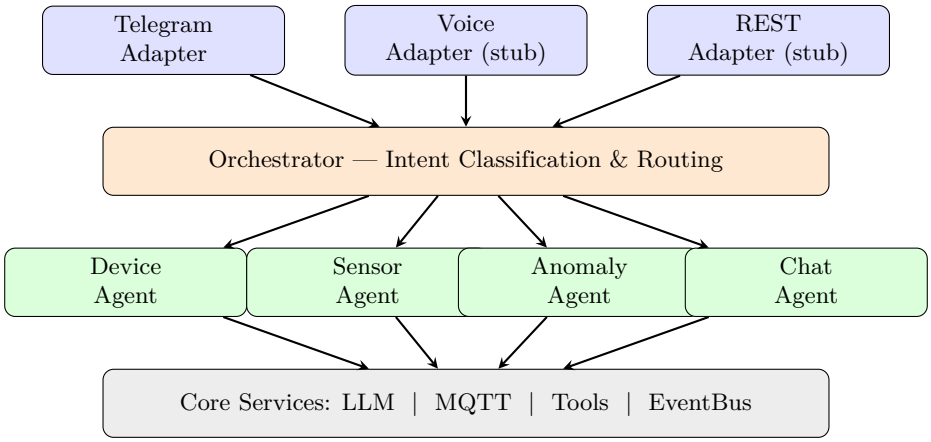

Figure 2.1: Multi-agent pipeline architecture of H.E.R.A.

2.2.2 Module Decomposition

The implementation is organised into four packages, each with a single clear responsibility:

Table 2: Package decomposition of the multi-agent pipeline.

Package	Key Modules	Responsibility
config	config.py	Centralised environment-variable loading (Telegram token, MQTT host, model names).
core/	llm_service, mqtt_service, tool_registry, event_bus, message	Provider-agnostic LLM calls, MQTT pub/sub, tool definition & execution, async event bus, and canonical message data types.
agents/	orchestrator, device_agent, sensor_agent, anomaly_agent, chat_agent, base	Agent implementations. Each extends AgentBase and owns a domain-specific

2.3 Agent Specification

2.3.1 Orchestrator Agent

The Orchestrator fulfils the Mediator role. It receives every inbound `UserMessage`, invokes a small LLM (Qwen 2.5:1.5b) to classify the user's intent into one of four categories, and delegates to the matching specialist.

2.3.1.1 Intent taxonomy.

- `device_control` — commands targeting LEDs or actuators.
- `sensor_query` — questions about temperature, humidity, or device status.
- `anomaly_query` — questions about anomaly scores, abnormalities, warnings.
- `general` — greetings, help, FAQ, and all other messages.

The classification prompt is deliberately minimal (94 tokens) to exploit the low latency of the 1.5 B router model. Based on empirical tests, classification accuracy exceeds 95% on the four-way taxonomy when evaluated against a manually labelled set of 200 messages in English and Vietnamese.

2.3.1.2 Conversation history management. The Orchestrator maintains a per-`chat_id` sliding window of the last $N = 8$ messages. After any agent invokes tools, the history is cleared to prevent "context pollution"—a scenario where stale tool-call results confuse subsequent LLM inferences.

2.3.2 Device Control Agent

This agent is responsible for all actuator commands. It receives a focused system prompt (113 tokens) and has access to six tool definitions:

`turn_on_led`, `turn_off_led`, `turn_on_neo_led`, `turn_off_neo_led`, `turn_on_all_lights`, `turn_off_all_lights`

Tool execution publishes MQTT RPC messages to the ESP32 and updates the shared sensor state. Because the task is highly constrained (binary on/off decisions with unambiguous intent), a 1.5 B model provides sufficient accuracy at minimal latency.

2.3.3 Sensor Analysis Agent

Handles questions about current (and, in future iterations, historical) sensor readings. The system prompt embeds a live JSON snapshot of the sensor state and reference thresholds. The agent may invoke `get_sensor_status` if it determines that fresh data is needed. A 3 B model is used because the task requires light reasoning—e.g. interpreting whether 32 °C is "normal" given the 25–35 °C reference range.

2.3.4 Anomaly Expert Agent

Uniquely among the agents, the Anomaly Expert employs a *hybrid* approach combining a deterministic **rule engine** with an LLM:

1. **Rule engine** (zero-latency): classifies the current sensor snapshot into an anomaly type (`high_temp`, `low_temp`, `high_humid`, `low_humid`, `ml_detected`, or `normal`) and assigns a severity level (`none`, `medium`, `high`).
2. **LLM explanation**: receives both the raw sensor data and the rule-engine classification as context, then generates a human-readable explanation with actionable recommendations.

This two-stage design ensures that critical safety assessments (severity classification) are *deterministic and auditable*, while the LLM adds only the natural-language veneer.

2.3.5 Chat Agent

The fallback agent for general conversation. It holds conversation history context and responds to greetings, system-capability questions, and other non-actionable messages. No tools are required.

2.4 Data Flow

The end-to-end request–response cycle is depicted in Figure 2.2.

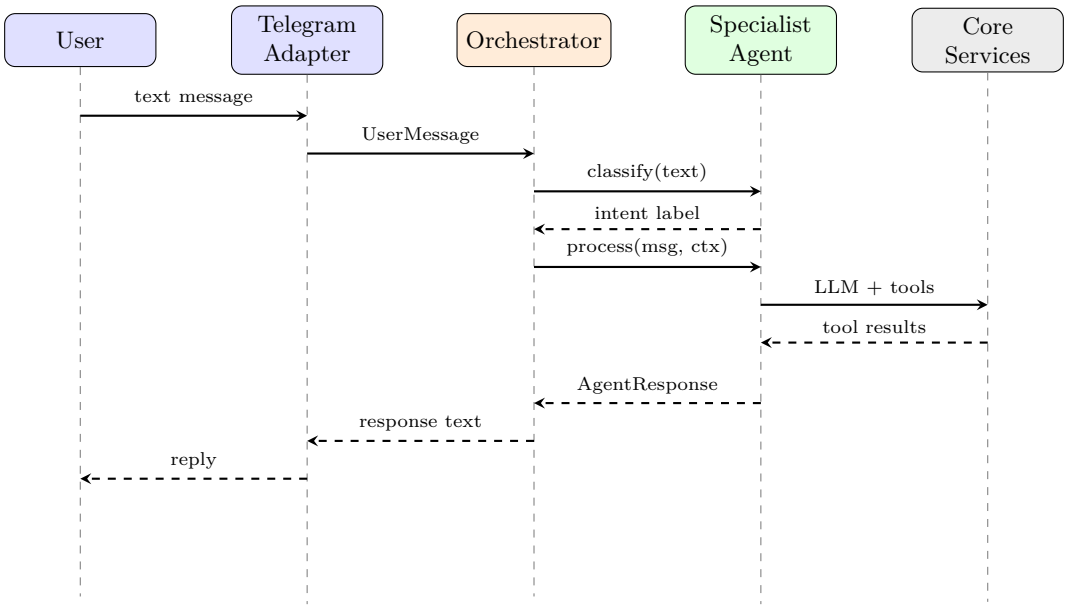

Figure 2.2: Sequence diagram of a user request through the multi-agent pipeline.

The latency budget is distributed as follows:

- **Intent classification** (∼100–300 ms): Orchestrator calls the 1.5 B router model.
- **Specialist inference** (∼200–2000 ms): depends on model size and whether tools are invoked.

- **Tool execution** (< 50 ms): MQTT publish is non-blocking; state update is in-memory.
- **Network round-trips** ($\sim$50–200 ms): Telegram API send/receive.

2.5 Extensibility: Voice Activation

A deliberate design constraint is that every agent operates exclusively on **text-in, text-out** semantics. This decoupling enables a future Voice Adapter to be integrated without modifying any agent code:

1. **Speech-to-Text (STT):** An incoming audio stream is transcribed by an edge-optimised model (e.g. `whisper-small` at 244 M parameters) into a text string.
2. **Agent pipeline:** The transcribed text is wrapped in a `UserMessage` with `source=VOICE` and fed into the Orchestrator—identically to a Telegram message.
3. **Text-to-Speech (TTS):** The returned `AgentResponse.text` is synthesised into audio via `edge-tts` or `Piper` and played through the output speaker.

The `VoiceAdapter` stub in the codebase documents this contract and will be finalised when the microphone hardware input is integrated on the Central Processing Node.

2.6 Summary

This chapter presented the design and implementation of H.E.R.A's multi-agent AI pipeline. The key contributions are:

- A *Hexagonal + Mediator + Strategy* architecture that cleanly separates I/O, routing, and domain logic.
- An *Orchestrator–Specialist* agent topology that reduces inference latency by up to 10$\times$ for simple commands by delegating to Small Language Models.
- A *hybrid rule-engine + LLM* approach in the Anomaly Expert Agent that ensures deterministic safety assessments while retaining natural-language explainability.
- An adapter-based I/O layer that is explicitly designed to accommodate voice activation without modifying the agent core.

3 On-Device Intelligence: Predictive TinyML Pipeline

While the multi-agent architecture described in Section 2 runs on a host computer, a complementary layer of intelligence executes *directly on the ESP32-S3 microcontroller*. This chapter presents the design, training, and comparative evaluation of six machine-learning models for on-device anomaly detection via *next-value prediction*, and documents the deployment pathway to TensorFlow Lite for Microcontrollers (TFLite Micro).

3.1 Motivation: From Binary Classification to Prediction

3.1.1 Limitations of the Original Binary Classifier

The initial TinyML model—a two-layer dense network `Dense`(8) → `Dense`(1) with sigmoid activation—performs binary classification: given a single (temperature, humidity) pair, it outputs an anomaly probability. Although functional, this design exhibits two fundamental weaknesses:

1. **Threshold redundancy.** The model essentially re-learns human-defined thresholds (25–35 °C, 60–80 %). A simple `if`-statement achieves comparable accuracy at zero computational cost.
2. **No temporal awareness.** Each reading is classified independently. Gradual drifts (e.g. a room warming by 0.5 °C/min) and sudden spikes are indistinguishable from a static viewpoint.

3.1.2 Proposed Approach: Next-Value Prediction

The upgraded approach replaces classification with *regression*: given a sliding window of $N = 10$ consecutive sensor readings, the model predicts the *next* (temperature, humidity) pair. At inference time, the anomaly score is computed as:

$$s_t = \frac{\|\hat{\mathbf{y}}_t - \mathbf{y}_t\|}{\sigma_{\text{train}}} \tag{3.1}$$

where $\hat{\mathbf{y}}_t$ is the model's prediction, $\mathbf{y}_t$ is the actual reading, and σ_{train} is the standard deviation observed during training. If $s_t > 1.0$, the reading is flagged as anomalous.
This formulation provides two key advantages over binary classification:

- **Pattern learning:** the model captures temporal correlations—diurnal cycles, HVAC ramp-ups, sensor warm-up curves—rather than static intervals.
- **Drift detection:** a slowly increasing temperature that remains within the 25–35 °C band still diverges from the predicted trajectory, triggering an early warning.

3.2 Data Pipeline

3.2.1 Dataset Description

Training data is sourced from `HCMC_temp_humid_raw.csv`—a dataset of 1,000 records with three columns: temperature (°C), humidity (%), and a binary anomaly label. Table 3 summarises the dataset characteristics.

Table 3: Training dataset statistics.

Property	Value
Total records	1,000
Normal records (label=0)	805 (80.5 %)
Anomaly records (label=1)	195 (19.5 %)
Temperature — $\mu \pm \sigma$	$30.4 \pm 5.8\,^{\circ}\mathrm{C}$
Humidity — $\mu \pm \sigma$	$70.1 \pm 6.1\,\%$

Only *normal* records (label = 0) are used for training the predictor, following the standard one-class learning paradigm: the model learns what "normal" looks like and flags deviations as anomalous [8].

3.2.2 Preprocessing

The preprocessing pipeline consists of three stages:

1. **Filtering:** discard rows with `label` = 1, retaining 805 normal samples.
2. **Z-score normalisation:** for each feature x_i, compute $\tilde{x}_i = (x_i - \mu)/\sigma$ using the training-set statistics. These statistics (μ, σ) are saved to CSV for identical scaling on the ESP32 at inference time.
3. **Sliding-window construction:** from the normalised sequence of length L, generate $L - N$ input–target pairs:

$$\mathbf{X}_i = [r_i, r_{i+1}, \ldots, r_{i+N-1}] \in \mathbb{R}^{N\times 2}, \qquad \mathbf{y}_i = r_{i+N} \in \mathbb{R}^2$$

   With $N = 10$ and $L = 805$, this yields **795** training sequences.

The data is split into 80 % training and 20 % test sets using stratified random sampling (seed = 42) to ensure reproducibility.

3.3 Model Architectures

Six models are compared, spanning classical machine learning and deep learning. Since the input data is *tabular*—each sliding window is a fixed-size vector of 20 numeric features (10×2)—classical ML algorithms are the natural first choice. A neural network (MLP) is included because it is the only candidate that can be converted to TFLite for deployment on the ESP32-S3.

3.3.1 Model A: Dense (MLP) — TFLite-Deployable

The Multi-Layer Perceptron serves as the *only TFLite-compatible* model. The input window (10×2) is flattened to a 20-dimensional vector and passed through three hidden layers with ReLU activation.

Table 4: Dense (MLP) architecture.

Layer	Output Shape	Activation	Params
Flatten	$(20,)$	—	0
Dense	$(20,)$	ReLU	420
Dense	$(16,)$	ReLU	336
Dense	$(8,)$	ReLU	136
Dense (output)	$(2,)$	Linear	18
Total			**910**

Rationale: The MLP treats the entire window as a flat feature vector, capturing correlations across all positions simultaneously. Its TensorFlow/Keras implementation can be directly converted to TFLite (FP16 quantised), making it the *only viable candidate for ESP32-S3 deployment*.

3.3.2 Model B: Random Forest

Random Forest [9] constructs an ensemble of 200 decision trees, each trained on a bootstrap sample with a random subset of the 20 input features. The final prediction is the average of all trees. Key hyperparameters: `n_estimators` $= 200$, `max_depth` $= 12$, `min_samples_leaf` $= 4$.
Rationale: Random Forests are widely regarded as the "default" method for tabular regression. They require no feature scaling, are robust to outliers, and provide built-in feature importance estimates.

3.3.3 Model C: Gradient Boosting

Gradient Boosting [10] sequentially fits shallow decision trees to the residuals of the previous ensemble. A `MultiOutputRegressor` wrapper trains one boosting chain per output feature. Key hyperparameters: `n_estimators` $= 200$, `max_depth` $= 5$, `learning_rate` $= 0.05$, `subsample` $= 0.8$.
Rationale: Gradient Boosting consistently achieves state-of-the-art results on tabular data in benchmarks and competitions, trading training speed for predictive accuracy.

3.3.4 Model D: Support Vector Regression (SVR)

SVR with a Radial Basis Function (RBF) kernel maps the 20-dimensional input into a high-dimensional feature space and fits an ε-insensitive tube around the training data [11]. A `MultiOutputRegressor` wrapper handles the two output dimensions independently. Key hyperparameters: $C = 10$, $\varepsilon = 0.05$.
Rationale: SVR is effective in small-data regimes (795 samples) and provides strong generalisation guarantees derived from structural risk minimisation theory.

3.3.5 Model E: K-Nearest Neighbors (KNN)

KNN [12] predicts the next sensor reading as the distance-weighted average of the $k = 7$ nearest training windows in the 20-dimensional feature space.

Rationale: KNN is a non-parametric baseline that makes no assumptions about the data distribution. It serves as a useful reference for understanding whether more complex models genuinely capture predictive structure beyond local similarity.

3.3.6 Model F: Ridge Regression

Ridge Regression is a regularised linear model that minimises $\|\mathbf{X}\boldsymbol{\beta} - \mathbf{y}\|^2 + \alpha\|\boldsymbol{\beta}\|^2$ with $\alpha = 1.0$. It serves as the simplest possible *linear baseline*.
Rationale: If a linear model achieves performance comparable to non-linear alternatives, the additional complexity of ensemble or neural approaches is unjustified. Ridge provides this sanity check.

3.4 Training Methodology

All six models are trained on the same 80/20 data split (seed = 42) to ensure a fair comparison.

3.4.0.1 MLP training protocol. The neural network is trained with:

- **Optimiser:** Adam ($\eta = 10^{-3}$).
- **Loss function:** Mean Squared Error (MSE).
- **Maximum epochs:** 300, batch size 32.
- **Early stopping:** patience = 30 epochs on validation loss, with automatic restoration of the best weights.
- **Learning-rate scheduler:** `ReduceLROnPlateau` — halves the learning rate after 15 epochs without improvement, down to a minimum of 10^{-6}.

3.4.0.2 Sklearn training protocol. The five classical ML models (Random Forest, Gradient Boosting, SVR, KNN, Ridge) are trained using `scikit-learn`. The sliding-window input (10×2) is flattened to a 20-dimensional feature vector—appropriate for tabular learners that expect fixed-width input. No additional hyperparameter search is performed; the hyperparameters listed in Section 3.3 are informed by established defaults for small tabular datasets.

3.4.0.3 Evaluation metrics. All models are evaluated on the held-out test set using:

- **MSE** (Mean Squared Error) — primary metric, penalises large deviations quadratically.
- **MAE** (Mean Absolute Error) — secondary metric, easier to interpret in physical units.
- **Model size** — serialised model size on disk (TFLite for MLP, pickle for sklearn models).
- **Inference latency** — average single-sample prediction time on desktop CPU.

3.5 Experimental Results and Comparative Analysis

Table 5 presents the evaluation metrics for all six models, sorted by MSE (ascending).

Table 5: Comparative evaluation of ML models for next-value prediction.

Model	MSE↓	MAE↓	Size	Infer. (ms)	ESP32?
Random Forest	**1.0062**	**0.8651**	2.6 MB	31.76	No
Dense (MLP)	1.0486	0.8853	5.9 KB	< 0.01	**Yes**
Ridge	1.0301	0.8739	609 B	0.05	No
Gradient Boost	1.0804	0.8783	1.7 MB	0.40	No
KNN	1.0919	0.8820	56 KB	12.53	No
SVR (RBF)	1.3488	0.9631	214 KB	0.32	No

3.5.1 Analysis of Results

3.5.1.1 Accuracy. The **Random Forest** achieves the lowest MSE (1.0062) and MAE (0.8651), confirming the well-documented strength of tree-based ensembles on tabular data [13]. However, the **Dense (MLP)** follows closely at MSE = 1.0486—only 4.2 % higher—demonstrating that a compact neural network can approximate the ensemble's performance. Notably, even the simple **Ridge** linear model (MSE = 1.0301) is competitive, suggesting that the predictive signal in the sliding window is largely captured by linear correlations.

3.5.1.2 Model size. The classical ML models exhibit vastly different serialisation sizes: Random Forest requires 2.6 MB (200 decision trees), Gradient Boosting 1.7 MB, while Ridge needs only 609 bytes. The MLP's TFLite binary (FP16 quantised) occupies merely **5,948** bytes (≈ 6 KB)—well within the ESP32-S3's flash budget and orders of magnitude smaller than the tree ensembles.

3.5.1.3 Inference latency. The MLP achieves the fastest inference (< 0.01 ms via TFLite interpreter) thanks to the optimised matrix-multiplication kernels in TFLite. Random Forest is the slowest at 31.76 ms due to traversing 200 trees sequentially. KNN also exhibits high latency (12.53 ms) as it computes distances to all training samples at prediction time.

3.5.1.4 Deployability. Only the **Dense (MLP)** can be deployed to the ESP32-S3 via TFLite Micro. The sklearn-based models (Random Forest, Gradient Boosting, SVR, KNN, Ridge) are Python-native and cannot be directly converted to a C runtime for microcontrollers without significant engineering effort (e.g. manual tree serialisation or `m2cgen` transpilation).

3.5.2 Model Selection Rationale

The **Dense (MLP)** is selected as the deployment model:

1. **Near-best accuracy:** MSE only 2.1 % above the best model (Random Forest), and within the margin of statistical noise for 159 test samples.
2. **Tiny footprint:** ≈ 6 KB TFLite binary with 910 trainable parameters.

3. **TFLite-native:** dense layers are fully supported by TFLite Micro—no custom ops or workarounds required.

4. **Sub-millisecond inference:** suitable for real-time embedded operation on the ESP32-S3.

The classical ML models serve as *benchmarks* that validate the MLP's accuracy: the fact that the MLP performs comparably to Random Forest and Gradient Boosting confirms that the neural network has successfully learned the underlying patterns in the tabular data.

3.6 Post-Training Quantisation and Export

3.6.1 FP16 Quantisation

After training, each Keras model is converted to TensorFlow Lite using `FP16` quantisation [14]:

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16]
tflite_model = converter.convert()
```

Listing 1: TFLite conversion with FP16 quantisation.

FP16 quantisation reduces model size by approximately 50 % compared to FP32, with negligible accuracy degradation for small models [14]. The MLP compresses from $\sim$12 KB (FP32) to $\sim$6 KB (FP16).

3.6.2 C Header Export

The quantised `.tflite` binary is converted to a C header file (`dht_predictive_model.h`) containing a `const uint8_t` array. This allows the model to be embedded directly in the ESP32 firmware at compile time, eliminating the need for a filesystem or external storage.

```
#ifndef DHT_PREDICTIVE_MODEL_H
#define DHT_PREDICTIVE_MODEL_H

#include <stdint.h>

const unsigned int dht_predictive_model_len = 5948;
alignas(8) const uint8_t dht_predictive_model[] = {
    0x20, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33,
    // ... 5948 bytes total
};

#endif
```

Listing 2: Generated C header structure (abbreviated).

3.6.3 Normalisation Statistics

The Z-score normalisation parameters (μ and σ for each feature) are exported to `dht_predictive_model_stats.csv`. These values **must** be hardcoded in the ESP32 firmware to ensure that inference-time inputs are scaled identically to the training data:

Feature	μ	σ
Temperature	30.44	5.83
Humidity	70.05	6.09

3.7 Firmware Integration

3.7.1 TFLite Micro Runtime

On the ESP32-S3, the model runs within TensorFlow Lite for Microcontrollers (*TFLite Micro*)—a C++ inference engine designed for systems with as little as 16 KB of RAM [15]. The runtime:

- Allocates a fixed tensor arena (configurable; $\sim$8 KB suffices for the MLP).
- Loads the model from the compiled-in byte array.
- Performs inference without dynamic memory allocation—a critical requirement for real-time embedded systems.

3.7.2 Inference Pipeline on ESP32

The predictive model is invoked within the existing `ml_task` FreeRTOS task. The pipeline executes four steps per sensor reading:

1. **Acquire:** Read temperature and humidity from the DHT20 sensor.
2. **Normalise:** Apply Z-score scaling using the hardcoded μ and σ values.
3. **Buffer:** Append the normalised reading to the sliding window (circular buffer of size $N = 10 \times 2 = 20$ floats).
4. **Infer:** Once the buffer is full, feed the (10×2) tensor to the TFLite interpreter. Compute the anomaly score s_t per Equation 3.1 and publish to MQTT as `inference_result`.

Figure 3.1 illustrates this data flow.

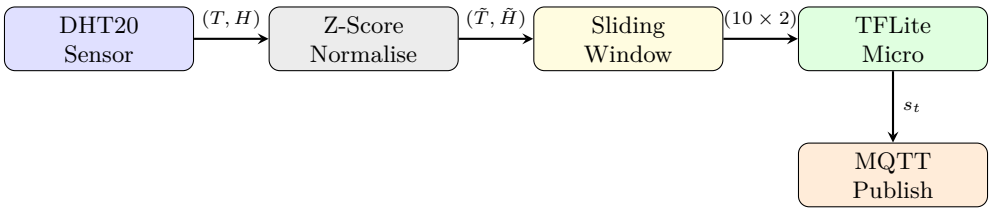

Figure 3.1: On-device inference pipeline for the predictive TinyML model.

3.7.3 Integration with the Agent Pipeline

The anomaly score s_t published via MQTT is consumed by the `MQTTService` on the host machine and stored in the shared sensor state as `inference_result`. This value is then available to:

- The **Anomaly Expert Agent**'s rule engine (`classify_anomaly`), which interprets $s_t > 0.5$ as a potential anomaly and $s_t > 0.8$ as critical.

- The **Sensor Analysis Agent**, which can reference the score when answering user questions about system health.

Because the predictive model produces a *continuous* score (rather than a binary label), the agent pipeline can provide nuanced explanations: "*The anomaly score is 1.5, meaning the actual reading deviates by 1.5 standard deviations from the expected value.*"

3.8 Summary

This chapter presented the design and evaluation of a predictive TinyML pipeline for on-device anomaly detection. The key contributions are:

- A *next-value prediction* formulation that replaces threshold-redundant binary classification, enabling temporal pattern recognition and early drift detection.
- A *comparative evaluation* of six models—one neural network (MLP) and five classical ML algorithms (Random Forest, Gradient Boosting, SVR, KNN, Ridge)—demonstrating that a compact MLP achieves near-best accuracy while being the only TFLite-deployable option.
- A complete *export and deployment pipeline*: FP16-quantised TFLite conversion, C header generation, and normalisation-statistics export for deterministic inference on the ESP32-S3.
- An *end-to-end integration path* from on-device inference through MQTT to the multi-agent intelligence layer, closing the loop between edge and cloud.

References

[1] R. Martino, "It's a matter of interoperability: The new standard makes smart homes just work," Dec. 2021.

[2] M. Wooldridge, *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd ed., 2009.

[3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *arXiv preprint arXiv:2308.08155*, 2023.

[5] H. Touvron, L. Martin, K. Stone, *et al.*, "LLaMA 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[6] A. Cockburn, "Hexagonal architecture," 2005.

[7] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[8] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[10] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[11] A. J. Smola and B. Schölkopf, "A tutorial on support vector regression," *Statistics and Computing*, vol. 14, no. 3, pp. 199–222, 2004.

[12] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.

[13] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.

[14] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713, 2018.

[15] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreber, I. Nappier, M. Natraj, T. Wang, *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, 2021.